

Driver's Office

F1 - TELEMETRY DASHBOARD

Software Design Document

Version 1.0 | May 2026

Author	Status	Reviewed By	Last Updated
Vishak Lakshman	DRAFT	Pending	May 2026

Module	Tech Stack	Cloud
Backend API	Python / FastAPI / Mangum	AWS Lambda + API Gateway
Frontend	Next.js 14 / React / Zustand	AWS Amplify (SSR)
Data Pipeline	Python / FastF1 / Pandas	AWS S3
Cache / Rate Limit	Redis (Upstash)	Serverless / Regional

1. Overview

This document describes the architecture, design decisions, security model, and future roadmap of the F1 Telemetry Dashboard a production-grade web application that transforms raw Formula 1 timing data into interactive, real-time visualisations for engineers, analysts, and enthusiasts.

1.1 Problem Statement

Official F1 timing data is available via the FastF1 Python library but is locked behind a heavy offline processing step (300MB+ session files, 60-second download latency). No publicly accessible tool exists that provides:

- Side-by-side driver telemetry comparison with synchronised animated replay
- Tyre degradation curves with statistical regression over a race stint
- Full race strategy visualisation (Gantt-style stint chart + pit stop log)

Existing solutions either require local Python environments (inaccessible to non-engineers) or are proprietary broadcast tools unavailable to the public.

1.2 Solution

A decoupled, serverless system with three distinct phases:

- Offline pipeline — processes and uploads session JSON to S3 once per race weekend
- Serverless API — Lambda reads from S3/Redis and serves lightweight JSON with no heavy dependencies
- SSR frontend — Next.js 14 App Router renders interactive dashboards with Framer Motion animations, Recharts data visualisations, and an SVG track map with animated ghost-car replay

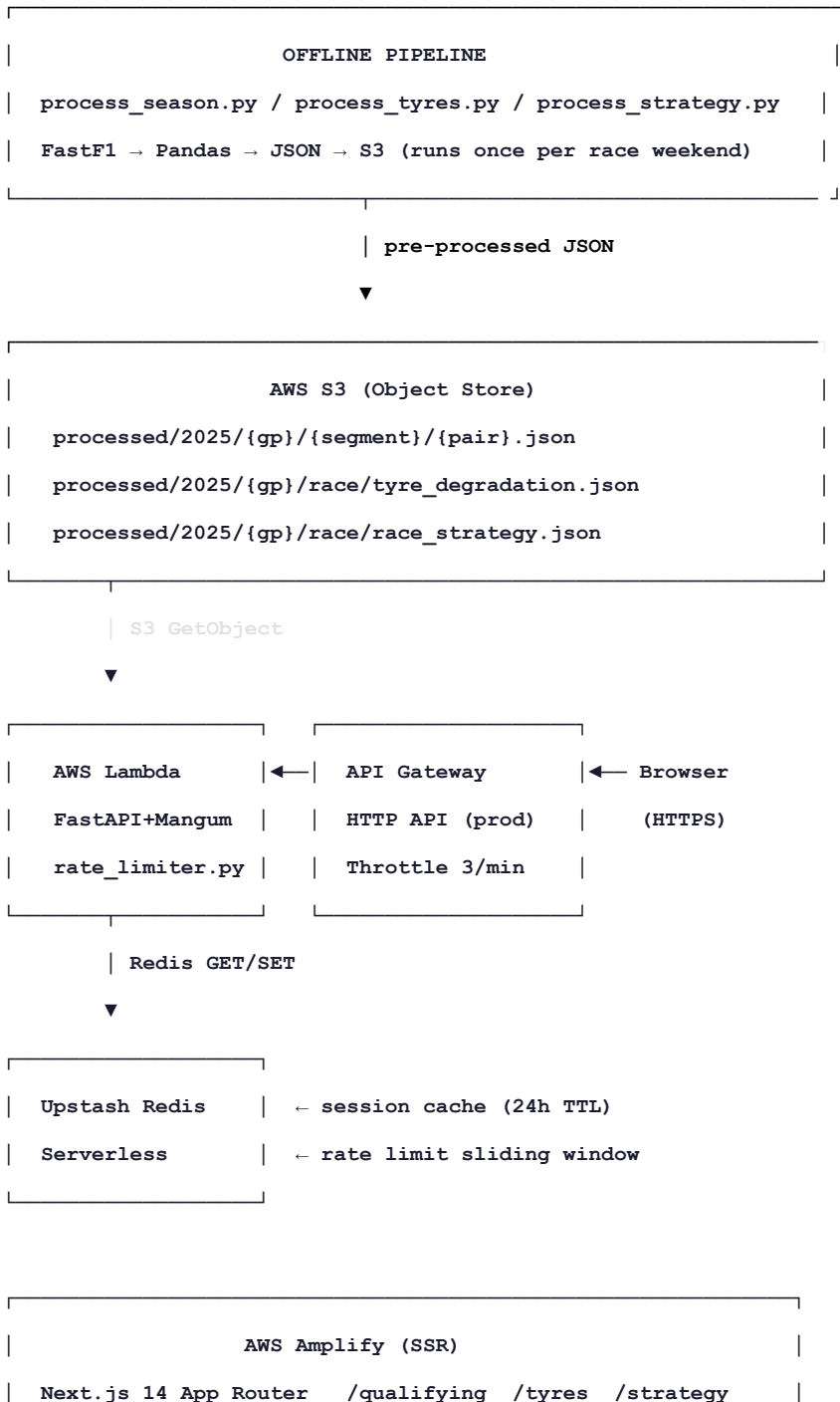
1.3 Non-Goals

- Real-time live race telemetry (FastF1 does not support live streaming)
- User authentication or per-user data persistence
- Mobile-native applications

2. System Architecture

2.1 High-Level Architecture

The system follows a Producer-Consumer pattern with three independently deployable layers:



| Zustand store · Recharts · SVG track map |

2.2 Data Flow

The system uses a three-tier caching strategy to ensure sub-100ms response times after the first request per dataset:

Tier	Store	Latency	Use Case
1	Upstash Redis	~5ms	Processed JSON (24h TTL)
2	AWS S3	~300ms	Pre-processed JSON (persistent)
3	FastF1 CDN	60–90s	Pipeline only — never at runtime

2.3 Request Lifecycle

The following describes a cache-miss request (first-ever load of a dataset):

1. Browser requests GET /api/qualifying/ghost?year=2025&gp=Monaco&session=Q3&driver1=VER&driver2=LEC
2. API Gateway checks throttle limit (0.05 req/s sustained). Rejects with HTTP 429 if exceeded.
3. Lambda cold-starts (~800ms) or reuses warm container. Mangum adapts the ASGI request.
4. Redis sliding-window middleware checks client IP rate limit (3 req/60s). Returns 429 with Retry-After header if exceeded.
5. Router calls `redis_service.get_cached(key)`. Cache miss → proceeds to S3.
6. `s3_service.get_ghost_lap_data()` calls S3 `GetObject` on `processed/2025/Monaco/Q3/LEC_VER.json` (~300ms).
7. Result stored in Redis (24h TTL). Response returned with `X-RateLimit-*` headers.
8. Subsequent identical requests return from Redis in ~5ms. `cached: true` in response body.

3. Component Design

3.1 Offline Pipeline

Three independent pipeline scripts run on the developer's machine after each race weekend. They share `upload_to_s3.py` and the same S3 bucket.

Script	Session Loaded	S3 Output
<code>process_season.py</code>	Qualifying (Q1/Q2/Q3 segments)	<code>{gp}/{segment}/{pair}.json</code>
<code>process_tyres.py</code>	Race (stint/compound data)	<code>{gp}/race/tyre_degradation.json</code>
<code>process_strategy.py</code>	Race (positions + pit stops)	<code>{gp}/race/race_strategy.json</code>

Key design decisions in the pipeline:

- `Session4DateUtc` (qualifying date) used for completed-race filtering, not `EventDate` (weekend start), to correctly handle sprint weekends and future events
- Out-laps and in-laps excluded from tyre degradation via `iloc[1:-1]` per stint rather than `PitInTime/PitOutTime` which are sparsely populated
- Linear regression slope (ms/lap) computed per stint using `numpy.polyfit` with a minimum of 3 data points
- Safety car laps filtered via `TrackStatus` column (codes 4/6/7) before degradation analysis
- Driver pairs stored with sorted codes (`LEC_VER` not `VER_LEC`) for canonical key lookup

3.2 Backend API

The backend is a FastAPI application running inside a Docker container on AWS Lambda via the Mangum ASGI adapter. All heavy dependencies (FastF1, Pandas, Scipy) are pipeline-only — the Lambda image contains only FastAPI, Mangum, boto3, and redis-py (~200MB vs the previous ~2GB).

Endpoint	Method	Description
<code>/health</code>	GET	Lambda health check (bypasses rate limit)
<code>/api/qualifying/session-info</code>	GET	Driver list for a qualifying session
<code>/api/qualifying/ghost</code>	GET	Telemetry + delta for a driver pair
<code>/api/tyres/degradation</code>	GET	Tyre stint + regression data for a race
<code>/api/strategy/race</code>	GET	Stint Gantt + positions + pit stops

3.3 Frontend

The frontend is a Next.js 14 App Router application hosted on AWS Amplify in SSR (Web Compute) mode. Global state is managed by Zustand — a single store shared across all three modules. All module data (ghost, tyre, strategy) and UI state (highlighted driver, playback position) live in one store, enabling cross-module synchronisation via the Global Header.

Route	Primary Component	Key Interaction
/qualifying	TrackMap.tsx (SVG)	Animated ghost-car replay with timeline scrubber
/tyres	TyreDegradationChart.tsx	Scatter + regression, driver filter pills
/strategy	StrategyGantt.tsx (SVG)	Hover highlights driver across all three sub-charts

4. Security Design

4.1 Threat Model

The primary threats for a public-facing read-only API are:

- Abuse / scraping — automated clients exhausting Lambda invocations
- Cost amplification — malicious traffic driving up AWS Lambda and S3 GET costs
- Data exfiltration — S3 bucket accessed directly without API
- Credential leakage — secrets exposed in source code or logs

4.2 Defence in Depth

Four independent security layers are implemented:

Layer 1 — API Gateway Throttling

AWS API Gateway enforces a hard request rate at the infrastructure level before any Lambda invocation occurs. This prevents cost amplification from Lambda cold-starts.

Parameter	Value	Effect
ThrottlingRateLimit	0.05 req/s	3 requests/minute sustained
ThrottlingBurstLimit	3	Max 3 concurrent burst requests

Layer 2 — Redis Sliding Window Rate Limiter

A per-IP sliding window is enforced inside the FastAPI middleware using Upstash Redis sorted sets. This is stateless across Lambda invocations — all instances share the same Redis state.

- Key: ratelimit:{client_ip}
- Scores: Unix timestamps of each request
- Window: 60 seconds, limit: 3 requests
- Pipeline operations (atomic): zremrangebyscore → zcard → zadd → expire
- Fail-open: if Redis is unreachable, requests are allowed to prevent a Redis outage from taking down the API
- Response headers: X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Window, Retry-After

Layer 3 — S3 Bucket Policy

The S3 bucket is fully private. All four public access block settings are enabled. The only entity with read access is the Lambda execution role via an IAM inline policy scoped to s3:GetObject, s3:PutObject, and s3:ListBucket on the specific bucket ARN. Direct public URL access returns 403 Forbidden.

Layer 4 — Secrets Management

No secrets exist in source code or environment files committed to Git. The following controls are in place:

- `.env` and `.env.local` are listed in `.gitignore`
- `UPSTASH_REDIS_URL` is passed as a Lambda environment variable via Python JSON serialisation (never shell-interpolated) to prevent special-character parsing vulnerabilities
- AWS credentials are stored only in `~/.aws/credentials` on developer machines and in IAM roles for Lambda (no access keys in Lambda)
- `ALLOWED_ORIGINS` is an explicit allowlist — wildcard (*) is never used

4.3 CORS Policy

Header	Value
Allow-Origins	Explicit list: localhost:3000, *.amplifyapp.com domain
Allow-Methods	GET, OPTIONS only — no mutation endpoints
Allow-Headers	Content-Type, Authorization
Allow-Credentials	true (scoped to explicit origins)

5. Module Specifications

Module A — Qualifying Ghost Session

Attribute	Detail
Purpose	Compare two drivers' fastest qualifying laps with animated side-by-side ghost replay
Data Source	FastF1 qualifying session telemetry (X/Y position + Speed/Throttle/Brake/Gear/DRS)
Alignment	Laps aligned by distance (metres), not time. Linear interpolation at 500 equidistant points.
Track Map	Raw (X,Y) normalised to 1000x1000 SVG viewBox with 50px padding. Animated dots interpolated from track_path distance array.
Delta Chart	Area chart (Recharts) showing time delta in seconds over lap distance. Positive = Driver 1 slower.
Playback	requestAnimationFrame loop in GlobalHeader. Progress drives all components simultaneously via Zustand progressPct (0-1).

Module B — Tyre Degradation Analysis

Attribute	Detail
Purpose	Visualise pace loss over a race stint per driver and compound
Filtering	Out-lap (first) and in-lap (last) dropped via <code>iloc[1:-1]</code> . SC/VSC laps dropped via <code>TrackStatus</code> (codes 4/6/7). Outliers > 120% of stint median marked <code>is_valid=false</code> .
Regression	Linear regression (<code>numpy.polyfit</code> degree 1) of <code>lap_time_ms</code> vs <code>tyre_life</code> . Slope reported as <code>ms/lap</code> . Minimum 3 valid data points required.
Compounds	Soft (<code>#FF3333</code>), Medium (<code>#FFD700</code>), Hard (<code>#FFFFFF</code>), Inter (<code>#39B54A</code>), Wet (<code>#0067FF</code>). Colours are F1 official palette.
UI	<code>ComposedChart</code> (Recharts) with Scatter points and dashed Line regression overlay per stint. Custom dots render outliers at 20% opacity.

Module C — Race Strategy Visualisation

Attribute	Detail
Purpose	Horizontal Gantt chart showing every driver's stint compound, duration, and pit stop timing

Gantt Render	Pure SVG — no charting library. ROW_H=32px per driver. Bars computed as $x=(start_lap/total_laps)*chartWidth$. Pit stop markers rendered as white vertical lines with triangle indicators.
Position Chart	Recharts LineChart with reversed Y-axis (P1 at top). One line per selected driver coloured by team. Opacity reduced to 0.2 for non-highlighted drivers.
Pit Stop Table	Sorted by lap number. Pit duration < 3s highlighted green (fast stop), > 5s highlighted red (slow stop). Compound badges colour-coded.
Cross-highlight	Hovering a driver in any component (Gantt / position chart / pit table) dims all other drivers across all three components simultaneously via Zustand highlightedDriver state.

6. Infrastructure & Deployment

6.1 AWS Services

Service	Config	Purpose	Est. Cost/mo
Lambda	512MB, 30s timeout	API runtime	~\$0.00 (free tier)
API Gateway	HTTP API, /prod stage	TLS termination, throttle	~\$0.00
ECR	~200MB image	Lambda container registry	~\$0.02
S3	~3.4GB (full season)	Pre-processed JSON store	~\$0.08
Amplify	Web Compute (SSR)	Next.js hosting + CDN	~\$0.00 (free tier)
Upstash Redis	Serverless, TLS	Cache + rate limiting	~\$0.00 (free tier)
Total			~\$0.10/month

6.2 Deployment Pipeline

Deployment is fully scripted via `infra/deploy.sh` and `git push`. No manual console steps are required after initial setup.

Trigger	Action	Command
Code change	Amplify auto-deploy	<code>git push origin main</code>
Backend change	ECR push + Lambda update	<code>./infra/deploy.sh</code>
New race weekend	S3 JSON upload	<code>python process_*.py</code>
Env var change	Lambda config update + Amplify rebuild	<code>aws amplify start-job ...</code>

6.3 CI/CD Considerations

The current deployment is developer-triggered. Recommended evolution for a production system:

- GitHub Actions workflow to run `process_season.py` automatically on race weekend Monday (race + 24h to ensure data availability in FastF1)
- Lambda container image build and push triggered on merge to main via GitHub Actions + OIDC (no long-lived AWS access keys)
- Amplify branch previews for pull requests
- Upstash Redis cache invalidation webhook after pipeline completes

7. Performance Characteristics

Scenario	P50 Latency	Notes
Redis cache hit	~5ms	All subsequent requests after first load
S3 fetch (warm Lambda)	~300ms	First request per session after pipeline
Lambda cold start	~800ms	200MB image, no heavy deps
Pipeline (single GP)	~5–10 min	FastF1 download + processing (offline)
Pipeline (full 2025 season)	~2–4 hours	24 GPs × 3 sessions, run once
Frontend ghost animation	60 fps	requestAnimationFrame, linear interpolation

The pre-processing architecture is the key performance decision. By moving all FastF1 computation offline, Lambda invocations are reduced to pure JSON retrieval. The 60-90 second first-request latency of the original on-demand architecture is completely eliminated for end users.

8. Future Improvements

8.1 Short Term (1–2 months)

- GitHub Actions automation: auto-run all three pipeline scripts every race Monday at 12:00 UTC
- Sprint weekend support: detect sprint format from FastF1 event schedule and load 'Sprint Qualifying' session correctly
- Sector time breakdown: decompose lap delta into Sector 1 / Sector 2 / Sector 3 contribution in the ghost module
- Weather overlay: overlay air/track temperature and rainfall flag on tyre degradation charts
- Driver comparison table: sortable table of all drivers' fastest lap times per segment alongside the Ghost module

8.2 Medium Term (3–6 months)

- User authentication: Cognito-backed login to support saved comparisons, bookmarks, and personal dashboards
- Multi-year comparison: extend pipeline year parameter to compare e.g. Monaco 2024 vs Monaco 2025 sector times
- Constructor championship view: aggregate stint strategy across all drivers of a team to infer team pit stop decision patterns
- WebSocket live updates: subscribe to OpenF1 live timing API during race weekends for real-time position chart updates
- CSV/PNG export: allow downloading a driver's telemetry trace or tyre degradation chart as a file

8.3 Long Term (6–12 months)

- ML-based tyre model: replace linear regression with a physics-informed neural network (thermal deg + rubber falloff) trained on multiple seasons
- Pit window predictor: given current tyre life and pace delta, compute optimal pit window for each strategy (1-stop vs 2-stop)
- Natural language query interface: 'Show me where Verstappen lost time to Leclerc in Monaco Q3 2025' using Claude API integration
- Mobile companion app: React Native wrapper with push notifications for race weekend pipeline completion
- Custom team dashboards: white-label API + configurable frontend for fan sites and engineering teams

8.4 Known Limitations

Limitation	Impact / Mitigation
FastF1 data availability	Some 2026 sessions lack pos_data in OpenF1 API. Pipeline validates and skips gracefully.
No live data	FastF1 is historical only. OpenF1 live API is a future integration target.
Single-region deployment	All resources in us-east-1. Multi-region would require S3 replication + regional Lambda.
Redis single point of failure	Rate limiter fails open. Cache miss falls back to S3 (300ms vs 5ms).
No authentication	Rate limiting is IP-based only. A VPN hop bypasses per-IP limit.

9. Appendix

9.1 File Structure

```
f1-telemetry/
├── backend/                                # FastAPI Lambda application
│   ├── app/
│   │   ├── main.py                        # App factory, CORS, rate limit middleware
│   │   ├── models/schemas.py            # Pydantic models (all three modules)
│   │   ├── routers/                      # qualifying.py tyres.py strategy.py
│   │   └── services/                     # s3_service.py redis_service.py rate_limiter.py
│   ├── Dockerfile                        # Lambda container image (~200MB)
│   └── requirements.txt                   # FastAPI Mangum boto3 redis pydantic
├── frontend/                              # Next.js 14 App Router
│   ├── app/
│   │   ├── layout.tsx                    # Root layout: Sidebar + GlobalHeader
│   │   ├── globals.css                   # Design system: dark theme CSS variables
│   │   ├── qualifying/page.tsx
│   │   ├── tyres/page.tsx
│   │   └── strategy/page.tsx
│   ├── components/
│   │   ├── TrackMap.tsx                  # SVG ghost car animation
│   │   ├── DeltaChart.tsx               # Area chart (Recharts)
│   │   ├── TelemetryPanel.tsx           # Speed/Throttle line charts
│   │   ├── TyreDegradationChart.tsx     # Scatter + regression
│   │   ├── StintSummary.tsx             # Per-driver stint cards
│   │   ├── StrategyGantt.tsx            # SVG Gantt chart
│   │   ├── PositionChart.tsx            # Line chart (Recharts)
│   │   └── PitStopTable.tsx             # Sortable pit log table
│   ├── lib/api.ts                        # Typed API client + RateLimitError
│   └── store/useF1Store.ts               # Zustand global state
├── pipeline/                              # Offline data processing (developer machine)
│   ├── process_season.py                 # Qualifying telemetry
│   ├── process_tyres.py                  # Race tyre degradation
│   ├── process_strategy.py               # Race strategy + pit stops
│   └── upload_to_s3.py                   # S3 upload helper
└── infra/
```

```
|   └─ deploy.sh           # Full deployment script
|   └─ lambda-policy.json  # IAM policy template
└─ amplify.yml             # Amplify monorepo build config
   └─ .gitignore
```

9.2 Key Dependencies

Package	Version	Purpose
fastf1	3.3.9	F1 session data (pipeline only)
fastapi	0.111.0	Async Python web framework
mangum	0.17.0	ASGI adapter for AWS Lambda
next	14.2.5	React SSR framework
zustand	4.5.4	Lightweight global state management
recharts	2.12.7	React charting library (delta, tyre, position)
framer-motion	11.3.8	Layout transition animations

9.3 Glossary

Term	Definition
Ghost session	Side-by-side animated comparison of two drivers' fastest laps on the same track map
Stint	A continuous run of laps on one set of tyres between pit stops
Tyre life	Number of laps the current tyre set has been used (including previous session use for used sets)
Degradation slope	Linear regression coefficient of lap time (ms) vs tyre life (laps). Positive = pace loss per lap.
Out-lap	First lap of a stint after leaving the pit lane. Excluded from analysis (tyre warming)
In-lap	Final lap of a stint before entering the pit lane. Excluded from analysis (driver saving tyres)
DXA	Document Extensible Attributes — unit used in DOCX XML (1440 DXA = 1 inch)
Sliding window	Rate limiting algorithm that counts requests within a rolling time window rather than a fixed interval